

ENGG2112 Multi-Disciplinary Engineering

Semester 2, 2026
Week 3

Professor T. J. Lim



THE UNIVERSITY OF
SYDNEY

Celebrating 175 years

CRICOS 00026A TEQSA PRV12057



What We're Covering Today

- What's a model in the engineering context?
- How are engineering models described? How did/do we learn the best model for a given system?
- What do we mean by “AI model”? How do AI models differ from non-AI models?
- What are the main concepts in machine learning or artificial intelligence?

Model in a General Sense

Dictionary definition:

1. A three-dimensional representation of a person or thing or of a proposed structure, typically on a smaller scale than the original. E.g. “a model of St Paul’s Cathedral”.
2. A thing used as an example to follow or imitate. E.g. “the project became a model for other schemes”.



Engineering Models

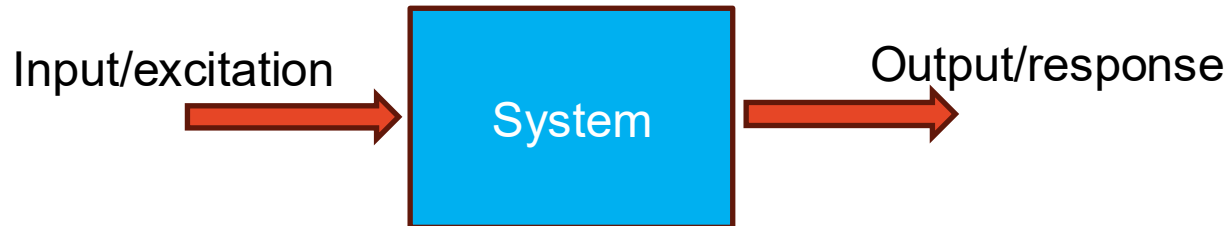
- These are closer to the first definition, i.e. **a representation of a thing**.
- This representation must be useful in engineering design.
- What are some examples you can think of?
- Probably model cars, buildings, and digital twins, all of which enable visualisation and some limited amount of design refinement and experimentation.

Engineering System Models

- Apart from models that represent a whole complex “thing” (e.g. a car, a building, a manufacturing process), engineers rely on models of **sub-systems** within that complex thing.
- These **models allow us to design** those sub-systems through computer simulations, mathematical analysis, lab experiments, etc. and ensure they work.
- Such models are necessarily mathematical:
 - A model for the human auditory system would be a filter that suppresses frequencies outside our hearing range and emphasizes and de-emphasizes some frequencies within that range.
 - A model for how flow rate into a hose impacts the force of water at the other end will account for the diameter of the nozzle.

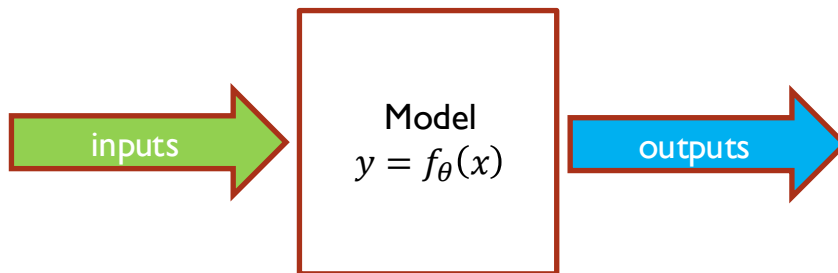
Input-Output Systems

- In engineering, a **system** takes in one or more **inputs** and produces one or more **outputs** in response to those inputs.
- A **deterministic system** always responds in the same way to a given input, whereas a random or **stochastic system** does not necessarily give the same output in response to a given input.
- What examples can you think of for deterministic and stochastic systems?
- Deterministic: Weighing scale or other measuring device, audio amplifier in a TV, car engine in response to gas pedal.
- Stochastic: Natural systems with inherent randomness, LLMs and other engineered systems that mimic natural randomness.



Input-Output Systems – General Mathematical Model

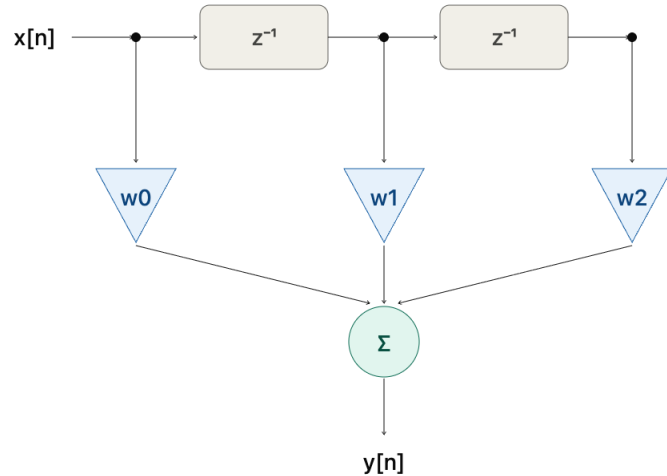
- From the first lecture, we already saw this model



$f_{\theta}(x)$ is a function (examples later) – x is usually multi-dimensional, the output of the model y may also be multi-D.

We need to find the best parameter θ for the system we're modelling – this is “training”.

Example 1: Discrete-Time Feedforward Model

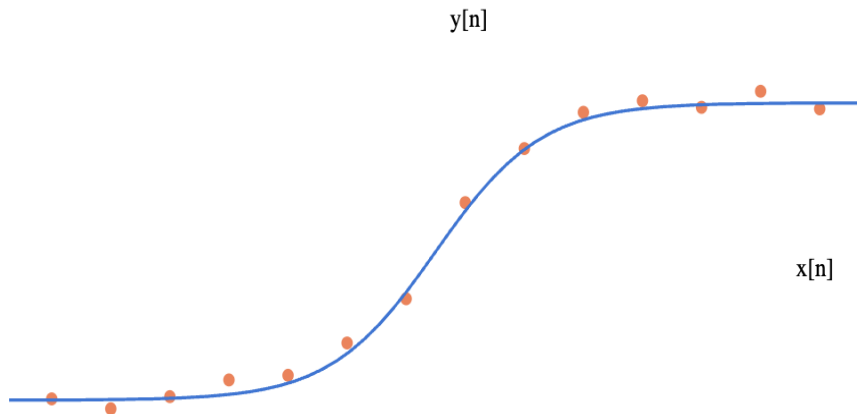


- In this model, $x[n]$ is a sequence of samples of a signal (e.g. audio sampled at 44.1 kHz); $y[n]$ are the samples of the output signal.
- We can design the weights w_0 , w_1 , w_2 to accomplish various functions, e.g. suppressing high frequencies.
- We can also use it to model an unknown input-output system.

$$y[n] = \sum_{k=0}^2 w_k x[n-k]$$

What is θ in this case?

Example 2: Non-Linear Model



*(There's an interactive demo in HTML
that is available on Canvas.)*

Let's model a memory-less power amplifier
using a hyperbolic tangent function:

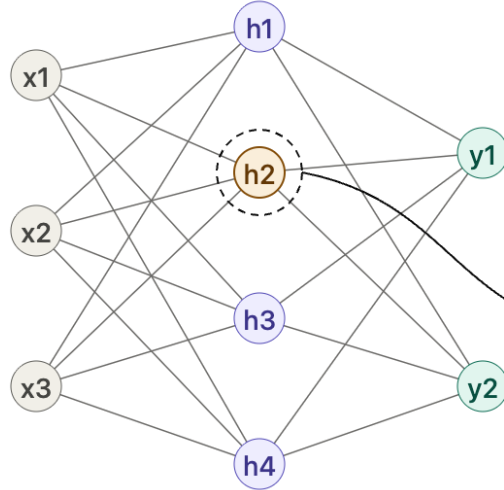
$$y[n] = A \cdot \tanh(g \cdot x[n])$$

We don't know the characteristics of this
power amplifier but we have the device and
can control its input and measure the
corresponding output.

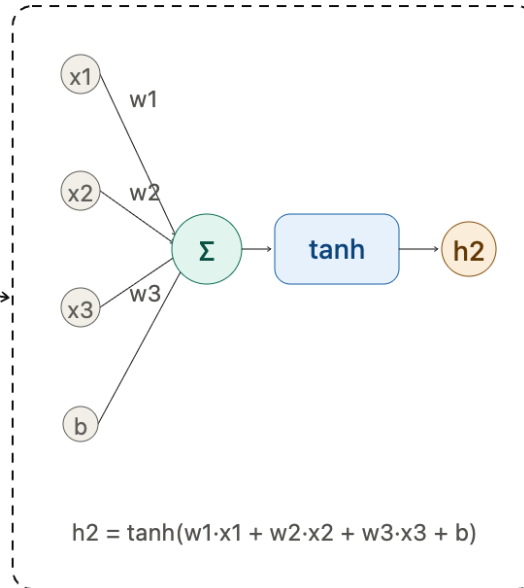
**What are the parameters of the model
that we need to learn?**

Example 3: Neural Network Model

input layer hidden layer output layer



Simplest NN model, a multi-layer perceptron (MLP)



What are the parameters of the model in this case?

How would we make the model more powerful?

How do we find the best weights in a given problem?

How Do LLMs Fit in the Above Picture?

Let's first watch this video together:

[Large Language Models explained briefly](#)

As you're watching, jot down the terms/concepts that you don't totally understand. Enter them into this Menti:



Basic Terminology

Data sets

Datasets

- Previously, we had introduced the model $y = f_{\theta}(x)$, where in the training phase the objective is to find the best value of θ using the training data available.
- Training data is organized into rows of $(x_1, \dots, x_K, y)$ data, assuming K scalar inputs (features) and one scalar output (response or label). These are often organized into CSV or Excel files, e.g.

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	Timestamp	Gearbox_T1_High_Speed_Shaft_Temperature	Gearbox_T3_High_Speed_Shaft_Temperature	Gearbox_T1_Intermediate_Speed_Shaft_Temperature	Temperature Gearbox Bearing Hollow Shaft	Tower Acceleration Normal	Gearbox_Oil-2_Temperature	Tower Acceleration Lateral	Temperature Bearing_A	Temperature Trafo-3	Gearbox_T1	Gearbox_T3	Gearbox_T1
2	1/01/2019 0:00	57	59	52	56.15833282	47.05377579	57	18.8907719	35	48.57666779	56.15833	55	55
3	1/01/2019 0:10	57.69333267	59.48500061	52.45000076	57	64.96974182	57.41333389	28.5602646	35.2333336	48	57	55.175	55.175
4	1/01/2019 0:20	59	60.75666809	53.53666687	57.77500153	51.14966965	58.72833252	34.2288132	36	48.05333328	57.775	56	56
5	1/01/2019 0:30	59.88166809	61.56333316	54.41333389	58.68333435	58.74092865	59.51833344	35.5932198	36	48.78833389	58.68333	56.73833	56.73833
6	1/01/2019 0:40	61.29000092	62.58666611	55.48500061	59.62333298	53.26477432	60.66500092	38.5527306	36	49	59.62333	57.615	57.615
7	1/01/2019 0:50	62.27166748	63.76833344	56.64500046	60.78333282	60.29292297	61.79833221	37.0367241	36.9016685	49.25999832	60.78333	58.70667	58.70667
8	1/01/2019 1:00	63	64.3666687	57.25666809	61.61333466	55.85303116	62.45333481	34.3804169	37	50	61.61333	59.555	59.555
9	1/01/2019 1:10	62.875	64.09333038	57.78333282	62	51.71955109	62.0616684	30.2212811	36.2666664	50	62	60	60
10	1/01/2019 1:20	63	64.94499969	58	62	48.2154541	63	37.6760826	35.0050011	50	62	60	60
11	1/01/2019 1:30	63.65833282	65.31666565	58.63499832	62.76499939	51.44218063	63.56499863	41.2617722	35.0116653	50	62.76333	60.67333	60.67333
12	1/01/2019 1:40	64.00666809	66	59	63.44499969	47.24278641	63.97000122	36.7862549	36	50	63.445	61.09	61.08833
13	1/01/2019 1:50	61.67499924	64.72666931	57.60833359	62.49833298	53.68341446	62.25833511	36.0348396	36	50	62.49833	59.87667	59.87667
14	1/01/2019 2:00	59.38999939	62.83833313	55.84999847	60.53333282	64.66885376	60.29166794	37.1026382	34.581665	50	60.53333	58.33667	58.33667
15	1/01/2019 2:10	59.56000137	62.03833389	55	60	61.39616776	60	60.4359703	34	50	60	58	58
16	1/01/2019 2:20	59.08000183	62	54.62333298	59.05333328	55.80208206	59.48333359	42.3691177	33.7833328	50.875	59.05333	57.23	57.23
17	1/01/2019 2:30	59.49166489	62	54.60833359	59	44.58469772	59.68500137	46.6666679	33.6133347	51	59	57	57

Datasets

- Each row is a **data record** or a **sample**.
- Depending on the problem some of the columns are **features**, and some are **responses** (or **labels**).
- Basically, we're interested in the machine **estimating** or **predicting** the label from the features.
- **Inference** is another word for estimation and prediction.

Labelled Data for Supervised Learning

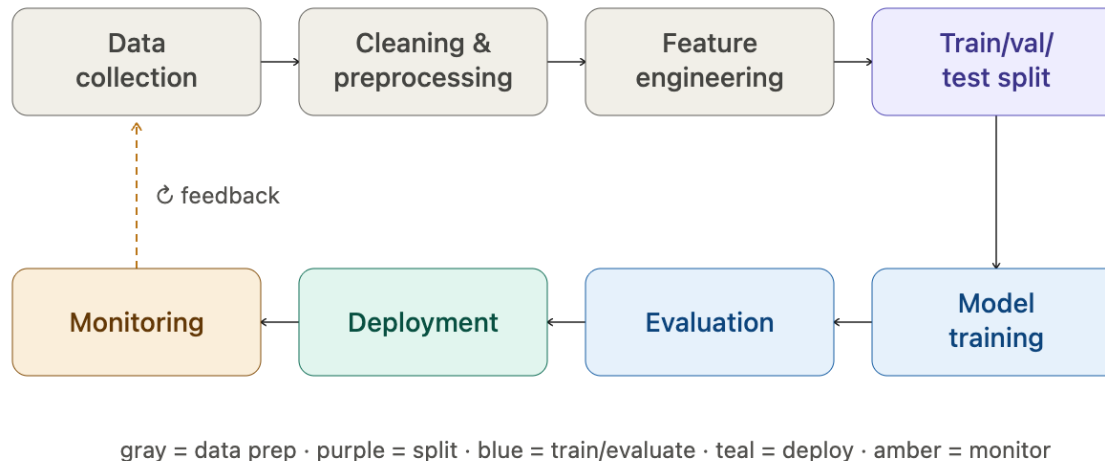
- In supervised learning, the ML model is trained on labelled data, i.e. each training data record has a true y value corresponding to the x values in that record.
- This true y value is known as the **ground truth** label.
- The x values are fed into the ML model – these are also known as **features** or **observations**.



The AI/ML Pipeline

End-to-end AI/ML data processing and deployment

Overview of the Pipeline



- AI/ML applications are often described in terms of a “pipeline” of functions, one leading into another.
- With the monitoring/feedback function, it's more of a cycle. We'll explain each part of the cycle next.

Data Preparation

- **Data Collection:** Need to collect the right data for the application. Remember “Garbage In, Garbage Out”.
 - In this unit, you probably won’t run experiments to collect your own data so you will rely on publicly available data sets.
 - Be sure to read the overview, description, etc. of the data set you’re downloading.
 - Academic papers often also provide their data sets freely. In that case, read the paper to understand what the authors did with the data.
- **Cleaning and Preprocessing:** Data may be missing, corrupted, in the wrong format, noisy, etc.
 - Usually need to do something to “clean up” the data before using it for training and testing.
 - More on this later.
- **Feature Engineering:** Either manually or automatically (through the AI architecture) extract the relevant attributes of the data for processing.

Data Split

- Think of a dataset as a spreadsheet with N rows and K columns
 - Each row is a sample; each column is a feature or a label.
 - E.g. in an image application, each sample represents one image in the dataset; the k -th column may be a pixel value corresponding to some position in the image; the final column may be a label such as “dog”, “car”, “bicycle”.
- **Training** requires some samples of data but not all.
 - If we use all samples for training, we have none left for **validation** – this is for deciding whether the current model is good or should be changed.
 - We also need some for **testing** – this is the final step where the final model is assessed on some other unseen data to see how well it generalizes.
- If we reuse data in training, validation or testing, there will be “leakage” from the validation and test sets into training, yielding better-than-normal results.
- Therefore, we split the available data into training/validation/test sets, e.g. 80/10/10.

Training & Evaluation

- Training an ML model is conceptually easy to understand, but mathematically intricate!
- Define a **loss function** that captures statistically how well the model is predicting the labels.
 - In regression, some variant of a **mean squared error (MSE)** function works;
 - In classification, need a **cross-entropy** function.
- The loss function must be differentiable with respect to the model parameters, θ .
- We then use some variant of **gradient descent** to minimize the loss function with respect to θ .
 - Loss function is defined in terms of statistical expectation, i.e. $E[\dots]$ but this is not knowable with precision.
 - Therefore, we use averages over the training samples as an approximation, leading to **stochastic gradient descent (SGD)**.
- *(See Appendix for some technical details, ask AI or your tutor to explain further if you're interested.)*

Training & Evaluation

- **Training set:** To adjust parameters for a given model and hyper-parameter setting.
- **Validation set:** To adjust hyper-parameter settings.
- **Testing set:** To evaluate performance of final model and settings; if unacceptable, need to re-design from the start, e.g. choose a different model or re-evaluate ML task.
- How do we evaluate performance in all of the above? We'll talk about that in a minute.
 - Accuracy, precision, recall, F1, etc.

Deployment & Monitoring

- AI application needs to be deployed
 - API, app on a phone, firmware on a microcontroller or DSP chip, etc.
- **Different constraints** from training/validation/testing in software
 - Memory, processing speed, real-time requirements
- **Mismatch** between training and deployment environments, e.g. sensors used in deployment may be noisier than those used in the lab for testing.
- Hence, **monitoring** of performance is critical after deployment.
 - If it doesn't work, we need to go back to the first step to re-design the system.
- Footnote: Contrary to popular belief, AI systems generally do not learn on-the-fly. The model is static after training and is re-trained periodically.

Classification, Regression, Performance Measures

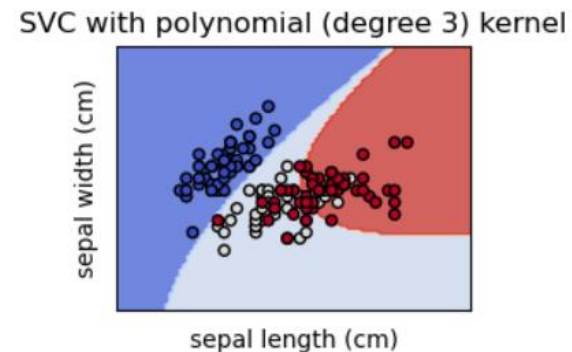
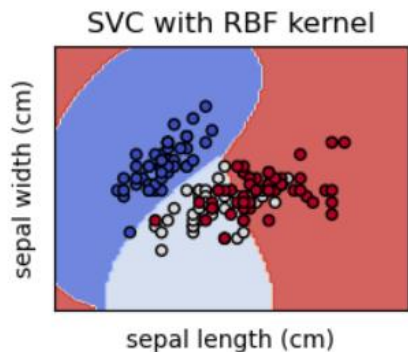
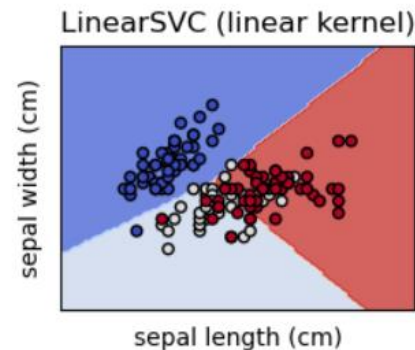
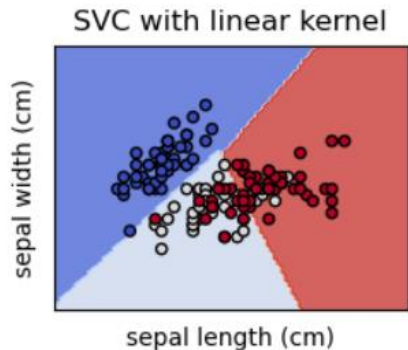
How do we evaluate performance? Does it depend on the ML task?

Classification Versus Regression

- There are two basic types of problems in ML: **Classification** and **Regression**.
- In classification, the set of all possible outputs, $\mathcal{S}_y$, is finite.
 - Problem is to decide which of a **finite set of classes** the observed features belong to, e.g. whether sensor readings indicate a problem, whether proximity sensors in a car indicate it should brake, whether a robot arm should pick up an object, etc.
- In regression, the set of possible outputs $\mathcal{S}_y$ is uncountable (and therefore infinite).
 - Problem is to **estimate the value** of the response to the inputs/features. E.g., predicting amount of rainfall tomorrow based on weather conditions today, estimating the traffic volume at 5pm given various related measurements at 4pm, estimating someone's age based on a photo, etc.
- In both cases, the model $y = f_{\theta}(x)$ applies. We note here that classification and regression machines are often capable of producing the output **probability distribution**.
 - This distribution is very useful – not just one output (*point estimate*) but a range of outputs with probabilities (*soft estimate*).

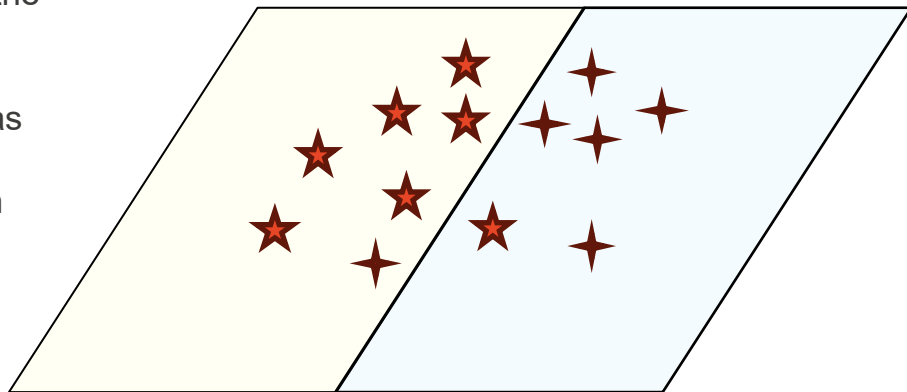
Observation Space and Decision Regions

- We can visualize input vectors as lying in a geometrical observation space.
- Think of a 2D slice of that multi-dimensional space and you get something like this →
- The classification task here is to **decide** which of three classes an observation belongs in.
- An ML model or any classifier will divide up the observation space into three non-overlapping decision regions, as illustrated here.



Classification Errors

- Classifiers can make errors – how often does your phone fail to recognize your face?
- Suppose the crosses belong in the blue class and the stars in the yellow class.
- Let the slanted line in the middle be the **decision boundary**, i.e. all points to the right are classified as “blue” and all to the left as “yellow”.
- Because the classifier is not perfect (perhaps it can only create decision boundaries that are straight lines) there will be errors in classification.
- In this example, 1 out of 6 crosses are incorrectly labelled as yellow, and 1 out of 7 stars as blue.
- How would we **quantify** the performance of this classifier?



Confusion Matrix and Performance Metrics

	Predicted positive	Predicted negative
Actual positive	TP True positive	FN False negative
Actual negative	FP False positive	TN True negative

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{FP} + \text{FN} + \text{TN})$$

$$\text{F1 score} = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$$

Example 1: Accuracy High But...

	Predicted positive	Predicted negative
Actual positive	TP True positive n = 5	FN False negative n = 45
Actual negative	FP False positive n = 10	TN True negative n = 940

$$\text{Precision} = 5 / (5 + 10) = 33\%$$

$$\text{Recall} = 5 / (5 + 45) = 10\%$$

$$\text{Accuracy} = (5 + 940) / 1000 = 94.5\%$$

$$\text{F1 score} = 2 \times 0.33 \times 0.10 / (0.33 + 0.10) \approx 15\%$$

- Here's a classifier that does a bad job but appears from its accuracy to do a terrific job – 94.5% accuracy!
- But precision = 33% means only 1/3 of those it predicted to be positive were actually positive.
- Recall = 10% means only 1 in 10 truly positive cases were detected.
- Problem is dataset is heavily skewed towards negative -- classifier can then output “negative” almost all the time, regardless of the input, and still achieve high accuracy.

Relative Cost of False Negative and False Positive

- In many cases, FN does not have the same cost as FP.
 - Medical diagnosis – FN means patient doesn't receive timely treatment; FP means patient goes for additional tests → FN is more costly in terms of risk to patient.
 - Intrusion detection – FN means a security breach is undetected and there may be major costs; FP means human experts are called in for further tests → FN is more costly.
 - Facial detection for phone unlocking – FN means you have to try again; FP means an unauthorized user is allowed to unlock your phone → FP is more costly.
- Therefore, depending on application, designer needs to look at the right performance metric.
- The F1 score is a harmonic mean of Recall and Precision, and balances the two. It can only be high if both are high.

Example 2: All Metrics Agree But...

	Predicted positive	Predicted negative
Actual positive	TP True positive n = 350	FN False negative n = 150
Actual negative	FP False positive n = 150	TN True negative n = 350

$$\text{Precision} = 350 / (350 + 150) = 70\%$$

$$\text{Recall} = 350 / (350 + 150) = 70\%$$

$$\text{Accuracy} = (350 + 350) / 1000 = 70\%$$

$$\text{F1 score} = 2 \times 0.70 \times 0.70 / (0.70 + 0.70) = 70\%$$

- Now consider a case of balanced data, 500 positive and 500 negative.
- Classifier correctly identifies positive cases and negative cases at a 70% rate.
- Recall, precision, accuracy, F1 are all 0.70.
- Is this good enough? Depends on application!
- This is an example of a classifier that is mediocre in both directions. Good candidate for re-design!

More Than Two Classes

		Predicted Class			
		A	B	C	D
Actual Class	A	85	2	10	3
	B	1	95	1	3
	C	12	1	82	5
	D	2	2	3	93

In non-binary classification, confusion matrix has this form.

Accuracy is defined as proportion of correct decisions (green).

Recall, precision and F1 are now defined on class-by-class basis, or “one-vs-rest”.

Regression Errors

- Unlike classification, errors in regression occur with probability one because the output of the ML model and the ground truth label are both continuous, so there is zero probability of the two being identical.
- Denoting the true label as y_0 and the model output as y , the error is $e = y - y_0$. Since we would like the error to be minimized, our goal is often to select the model and the corresponding θ to minimize the **mean squared error (MSE)**:

$$\zeta = E[(y - y_0)^2]$$

- Here, $E[\cdot]$ denotes the statistical mean operator. Just think of this as the average/mean that you already know.
- Unlike the classification task, regression tasks generally use only the MSE as a performance measure.

What's Coming Up?

Next Week's Class

Week 4 Preview

- Data pre-processing including one-hot encoding
- “Classical” ML models
 - Linear Regression
 - Naïve Bayes, Random Forest, Logistic Regression, etc.
 - How to implement them in Python
- Softmax function

See you next week!